

# 2D Array Operations Using a Menu-Driven C Program

---

## Aim

To write a menu-driven C program to perform different operations on a given 2D array (matrix) using switch-case statements, arrays, and loops. The program finds the sum of all elements, finds and stores each row sum in a 1D array, finds the transpose, and exits through a menu option.

## Algorithm

1. Start the program.
2. Input the number of rows and columns of the matrix.
3. Input all elements of the matrix using nested loops.
4. Display the menu: Sum of all elements, Sum of each row, Transpose of matrix, and Exit.
5. Accept the user choice and use a switch statement to perform the selected operation.
6. For Sum of All Elements, initialize sum to 0, traverse the matrix, add every element, and display the total.
7. For Sum of Each Row, calculate each row sum, store it in a 1D array, and display the individual sums and the row-sum array.
8. For Transpose, interchange the row and column positions while displaying the matrix.
9. For Exit, terminate the program.
10. Display an error message for an invalid menu choice and continue until Exit is selected.

## C Program

```
#include <stdio.h>

int main()
{
    int rows, cols, i, j, choice;
    int sum, rowSum[100];

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    int matrix[rows][cols];

    printf("Enter the matrix elements:\n");
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", &matrix[i][j]);

    do
    {
        printf("\n----- MATRIX OPERATIONS MENU ----- \n");
        printf("1. Sum of all elements\n");
        printf("2. Sum of each row\n");
        printf("3. Transpose of matrix\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
```

```

        sum = 0;
        for (i = 0; i < rows; i++)
            for (j = 0; j < cols; j++)
                sum += matrix[i][j];
        printf("Sum of all elements = %d\n", sum);
        break;

    case 2:
        printf("Row sums:\n");
        for (i = 0; i < rows; i++)
        {
            sum = 0;
            for (j = 0; j < cols; j++)
                sum += matrix[i][j];
            rowSum[i] = sum;
            printf("Sum of row %d = %d\n", i + 1, rowSum[i]);
        }
        printf("The row-sum array contains: [");
        for (i = 0; i < rows; i++)
        {
            printf("%d", rowSum[i]);
            if (i < rows - 1) printf(", ");
        }
        printf("]\n");
        break;

    case 3:
        printf("Transpose of the matrix:\n");
        for (j = 0; j < cols; j++)
        {
            for (i = 0; i < rows; i++)
                printf("%d\t", matrix[i][j]);
            printf("\n");
        }
        break;

    case 4:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice! Please enter a number from 1 to 4.\n");
    }
} while (choice != 4);

return 0;
}

```

## Test Case Table

| Test Case | Operation           | Input Matrix  | Expected Result        |
|-----------|---------------------|---------------|------------------------|
| 1         | Sum of all elements | 1 2 3 / 4 5 6 | Sum = 21               |
| 2         | Sum of each row     | 1 2 3 / 4 5 6 | Row sums = [6, 15]     |
| 3         | Transpose           | 1 2 3 / 4 5 6 | 1 4 / 2 5 / 3 6        |
| 4         | Sum of all elements | 2 4 / 6 8     | Sum = 20               |
| 5         | Sum of each row     | 2 4 / 6 8     | Row sums = [6, 14]     |
| 6         | Invalid choice      | Choice = 5    | Invalid choice message |
| 7         | Exit                | Choice = 4    | Program exits          |

## Expected Output

```
Enter the number of rows and columns: 2 3
Enter the matrix elements:
1 2 3
4 5 6

Enter your choice: 1
Sum of all elements = 21

Enter your choice: 2
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]

Enter your choice: 3
Transpose of the matrix:
1    4
2    5
3    6
```

## Result

The menu-driven C program was successfully implemented to perform the required 2D array operations. It calculates the sum of all matrix elements, calculates and stores the sum of each row in a 1D array, and displays the transpose of the matrix. The program uses 2D arrays, a 1D array, nested loops, and a switch-case statement to perform the required operations.